

Polimorfismo

La definición de polimorfismo que aparece en el diccionario hace referencia a un principio de la biología según el cual un organismo o una especie puede tener muchas formas o etapas diferentes. Este principio también se puede aplicar a la programación orientada a objetos y a lenguajes como Java. Las subclases de una clase pueden definir sus propios comportamientos únicos y, sin embargo, compartir algunas de las mismas funciones de la clase principal. El polimorfismo se puede demostrar con una pequeña modificación de la clase `Bicycle`. Por ejemplo, se podría añadir un método `printDescription()` a la clase que muestre todos los datos almacenados actualmente en una instancia.

```
1 public void printDescription(){
2     System.out.println("\nBike is " + "in gear " + this.gear
3         + " with a cadence of " + this.cadence +
4         " and travelling at a speed of " + this.speed + ". ");
5 }
```

Para demostrar las características polimórficas en el lenguaje Java, extienda la clase `Bicycle` con una clase `MountainBike` y una clase `RoadBike`. Para `MountainBike`, agregue un campo para la suspensión, que es un valor `String` que indica si la bicicleta tiene un amortiguador delantero, `Front`. O bien, la bicicleta tiene un amortiguador delantero y trasero, `Dual`.

Aquí está la clase actualizada:

```
1 public class MountainBike extends Bicycle {
2     private String suspension;
3
4     public MountainBike(
5         int startCadence,
6         int startSpeed,
7         int startGear,
8         String suspensionType){
9         super(startCadence,
10             startSpeed,
11             startGear);
12         this.setSuspension(suspensionType);
13     }
14
15     public String getSuspension(){
16         return this.suspension;
17     }
18
19     public void setSuspension(String suspensionType) {
20         this.suspension = suspensionType;
21     }
22
23     public void printDescription() {
24         super.printDescription();
25         System.out.println("The " + "MountainBike has a" +
26             getSuspension() + " suspension.");
27     }
28 }
```

Tenga en cuenta el método anulado `printDescription()`. Además de la información proporcionada anteriormente, se incluyen en la salida datos adicionales sobre la suspensión.

A continuación, crea la clase `RoadBike`. Dado que las bicicletas de carretera o de competición tienen neumáticos delgados, añade un atributo para realizar un seguimiento del ancho de los neumáticos. Esta es la clase `RoadBike`:

```
1 public class RoadBike extends Bicycle{
2     // In millimeters (mm)
3     private int tireWidth;
4
5     public RoadBike(int startCadence,
6                     int startSpeed,
7                     int startGear,
8                     int newTireWidth){
9         super(startCadence,
10              startSpeed,
11              startGear);
12         this.setTireWidth(newTireWidth);
13     }
14
15     public int getTireWidth(){
16         return this.tireWidth;
17     }
18
19     public void setTireWidth(int newTireWidth){
20         this.tireWidth = newTireWidth;
21     }
22
23     public void printDescription(){
24         super.printDescription();
25         System.out.println("The RoadBike" + " has " + getTireWidth() +
26                             " MM tires.");
27     }
28 }
```

Tenga en cuenta que, una vez más, se ha anulado el método `printDescription()`. Esta vez, se muestra información sobre el ancho del neumático.

En resumen, hay tres clases: `Bicycle`, `MountainBike`, y `RoadBike`. Las dos subclases anulan el método `printDescription()` e imprimen información única.

A continuación, se muestra un programa de prueba que crea tres variables `Bicycle`. Cada variable se asigna a una de las tres clases de bicicletas. Luego, se imprime cada variable.

```
1 public class TestBikes {
2     public static void main(String[] args){
3         Bicycle bike01, bike02, bike03;
4
5         bike01 = new Bicycle(20, 10, 1);
6         bike02 = new MountainBike(20, 10, 5, "Dual");
7         bike03 = new RoadBike(40, 20, 8, 23);
8
9         bike01.printDescription();
10        bike02.printDescription();
11        bike03.printDescription();
12    }
13 }
```

El siguiente es el resultado del programa de prueba:

```
1 | Bike is in gear 1 with a cadence of 20 and travelling at a speed of 10.  
2 |  
3 | Bike is in gear 5 with a cadence of 20 and travelling at a speed of 10.  
4 | The MountainBike has a Dual suspension.  
5 |  
6 | Bike is in gear 8 with a cadence of 40 and travelling at a speed of 20.  
7 | The RoadBike has 23 MM tires.
```

La máquina virtual Java (JVM) llama al método apropiado para el objeto al que se hace referencia en cada variable. No llama al método definido por el tipo de la variable. Este comportamiento se conoce como invocación de método virtual y demuestra un aspecto de las importantes características del polimorfismo en el lenguaje Java.

Ocultar campos

Dentro de una clase, un campo que tiene el mismo nombre que un campo de la superclase oculta el campo de la superclase, incluso si sus tipos son diferentes. Dentro de la subclase, no se puede hacer referencia al campo de la superclase por su nombre simple. En cambio, se debe acceder al campo a través de `super`, lo cual se explica en la siguiente sección. En términos generales, no recomendamos ocultar campos, ya que dificulta la lectura del código.

Usando la palabra clave Super

Acceso a miembros de superclase

Si su método reemplaza uno de los métodos de su superclase, puede invocar el método reemplazado mediante el uso de la palabra clave `super`. También puede usar `super` para hacer referencia a un campo oculto (aunque no se recomienda ocultar campos). Considere esta clase, `Superclass`:

```
1 | public class Superclass {  
2 |  
3 |     public void printMethod() {  
4 |         System.out.println("Printed in Superclass.");  
5 |     }  
6 | }
```

Aquí hay una subclase, llamada `Subclass`, que anula `printMethod()`:

```
1 | public class Subclass extends Superclass {  
2 |  
3 |     // overrides printMethod in Superclass  
4 |     public void printMethod() {  
5 |         super.printMethod();  
6 |         System.out.println("Printed in Subclass");  
7 |     }  
8 |     public static void main(String[] args) {  
9 |         Subclass s = new Subclass();  
10 |        s.printMethod();  
11 |    }  
12 | }
```

Dentro de Subclass, el nombre simple `printMethod()` hace referencia al declarado en Subclass, que reemplaza al de Superclass. Por lo tanto, para hacer referencia a `printMethod()` heredado de Superclass, Subclass debe utilizar un nombre calificado, utilizando `super` como se muestra. Al compilar y ejecutar Subclass se imprime lo siguiente:

```
1 Printed in Superclass.  
2 Printed in Subclass
```

Constructores de subclases

El siguiente ejemplo ilustra cómo utilizar la palabra clave `super` para invocar el constructor de una superclase. Recuerde que en el ejemplo Bicycle anterior MountainBike se trataba de una subclase de Bicycle. Aquí se muestra el constructor (subclase) MountainBike que llama al constructor de la superclase y luego agrega su propio código de inicialización:

```
1 public MountainBike(int startHeight,  
2                     int startCadence,  
3                     int startSpeed,  
4                     int startGear) {  
5     super(startCadence, startSpeed, startGear);  
6     seatHeight = startHeight;  
7 }
```

La invocación de un constructor de superclase debe ser la primera línea del constructor de subclase. La sintaxis para llamar a un constructor de superclase es

```
1 super();
```

O

```
1 super(parameter list);
```

Con `super()`, se llama al constructor sin argumentos de la superclase. Con `super(parameter list)`, se llama al constructor de la superclase con una lista de parámetros coincidentes.

Nota: Si un constructor no invoca explícitamente un constructor de superclase, el compilador de Java inserta automáticamente una llamada al constructor sin argumentos de la superclase. Si la superclase no tiene un constructor sin argumentos, obtendrá un error en tiempo de compilación. `Object` sí tiene un constructor de este tipo, por lo que, si `Object` es la única superclase, no hay problema.

Si un constructor de subclase invoca un constructor de su superclase, ya sea de forma explícita o implícita, se podría pensar que se invocará toda una cadena de constructores, hasta llegar al constructor de `Object`. De hecho, este es el caso. Se denomina *encadenamiento de constructores* y es necesario tenerlo en cuenta cuando hay una larga línea de descendencia de clases.

Escritura de clases y métodos finales

Puedes declarar algunos o todos los métodos de una clase como finales. Utiliza la palabra clave `final` en una declaración de método para indicar que las subclases no pueden anular el método. La clase `Object` hace esto: varios de sus métodos son finales.

Es posible que desees hacer que un método sea final si tiene una implementación que no se debe cambiar y es fundamental para el estado consistente del objeto. Por ejemplo, es posible que desees hacer que el método `getFirstPlayer()` en esta clase `ChessAlgorithm` sea final:

```
1  class ChessAlgorithm {  
2      enum ChessPlayer { WHITE, BLACK }  
3      ...  
4      final ChessPlayer getFirstPlayer() {  
5          return ChessPlayer.WHITE;  
6      }  
7      ...  
8  }
```

Los métodos invocados desde constructores generalmente deben declararse como finales. Si un constructor invoca un método no final, una subclase puede redefinir ese método con resultados sorprendentes o indeseables.

Tenga en cuenta que también puede declarar una clase completa como final. Una clase que se declara final no puede tener subclases. Esto es particularmente útil, por ejemplo, al crear una clase inmutable como la clase `String`.

Bibliografía: <https://dev.java/learn/inheritance/polymorphism/>